Distributed Job Scheduling Cluster

Fault-Tolerant, Load-Balanced Execution Infrastructure

Distributed Systems Project Team

Politecnico di Milano
090950 – Distributed Systems (A.Y. 2025–2026)
Professors: G. Cugola, A. Margara

October 28, 2026

Project Requirements & Goals

Distributed Job Scheduling

The system manages the submission, scheduling and execution of generic computational jobs over a cluster of executors.
The goal is to make the cluster behave as a single reliable job scheduler.

1. Job Submission

- Client contacts **any executor**.
- Job receives a unique `jobId`.
- Result can be retrieved asynchronously.

2. Load Sharing

- Jobs are dynamically assigned.
- Leader selects the **least-loaded** executor to achieve balance.

3. Fault Tolerance

- Leader and worker failures handled automatically.
- Persistent replicated state for consistency and recovery.

Communication and Discovery

Why Java RMI? (Point-to-Point RPC)

Dedicated to reliable, stateful one-to-one communication:

- **Code Mobility:** Transfers arbitrary computational tasks (`SerializableSupplier<T>`) directly across the network as serialized byte streams.
- **Return Values:** Operations requiring synchronous acknowledgements, return values, and explicit exception handling (ex: `submitJob`, `executeJob`).

Why UDP Multicast?

Dedicated to multicast communication: Each node sends heartbeat to each with information about the term and the current leader.

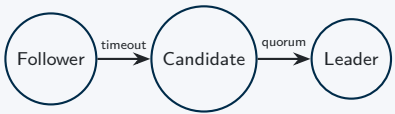
Architectural Synergy

UDP Multicast answers: *«Who is currently alive and reachable?»* (Decentralized, best-effort, zero-overhead)
Java RMI answers: *«What computation must be coordinated and executed?»* (Reliable, stateful, code-mobile)

Election Protocol

Node State

Each node can be:



Implemented Raft Mechanisms

When the failure of the leader is detected, the election mechanism started:

- Persistent `currentTerm`.
- Persistent `votedFor`.
- Randomized election timeout: **5–10 s**.
- `RequestVote` with:
  - candidate term,
  - candidate ID,
  - last log index,
  - last log term.
- Check that log is up-to-date.

Raft provides the consistency; it does not execute the jobs itself.

Raft Consensus: Proposal & Distribution

From Client Ingress to Cluster-Wide Propagation

Every state mutation begins as an uncommitted proposal at the Leader, which validates, persists, and distributes the command across the cluster.

1. **Client Request**
   - The Leader tracks unique client and request identifiers to detect retries, ensuring that a completed commited job never execute twice.
   - The client waits for cluster consensus.
2. **Parallel Log Distribution & Consistency Verification**
   - **Stable Storage Durability:** The operation is packaged into a new log entry and written locally before involving other nodes.
   - The Leader, after saves it locally, broadcasts uncommitted entries to all Followers concurrently via point-to-point RPC.
   - **Log Matching Property:** Each Follower verifies continuity with its previous entries. If terms or indices diverge, conflicting uncommitted entries from past leaders are deleted.
   - Once consistency is verified, Followers append the records directly to their persistent disk logs.

Raft Consensus: Quorum Commit & State Machine

From Majority Acknowledgement to Replicated State

Consensus is reached when a strict majority confirms log consistency, turning a tentative proposal into a permanently committed state transition.

3. Majority Quorum Verification & Commit Advancement

- Followers acknowledge successful log consistency back to the Leader.
- An entry is officially **committed** once acknowledged by a strict majority of the cluster:

$$\text{Replicas} \geq \left\lfloor \frac{\texttt{clusterSize}}{2} \right\rfloor + 1 \quad \wedge \quad \text{Entry Term} = \text{Current Term}$$

- The Leader after the commit return the job_id to the client as a confirmation of the correct reception of the request.

4. Deterministic State Machine Application

- Committed entries are applied sequentially to the local **Replicated State Machine** in identical order across all nodes. At this moment the job officially enters the active scheduling state.

Persistent Log vs In-Memory State Machine

RaftLog (Disk Storage)

Append-only log of state transitions, ensuring persistence across crashes.

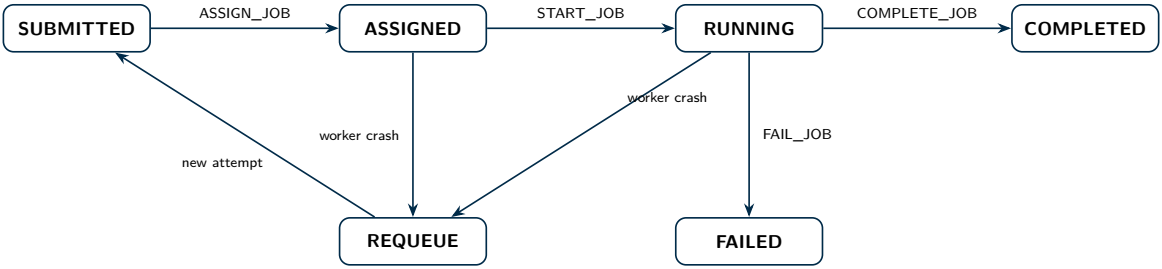
- Replicated across all cluster nodes.
- Saved persistently on disk: `<nodeId>_raft_log.dat`
- Tracks `commitIndex` (quorum reached).

JobStateMachine

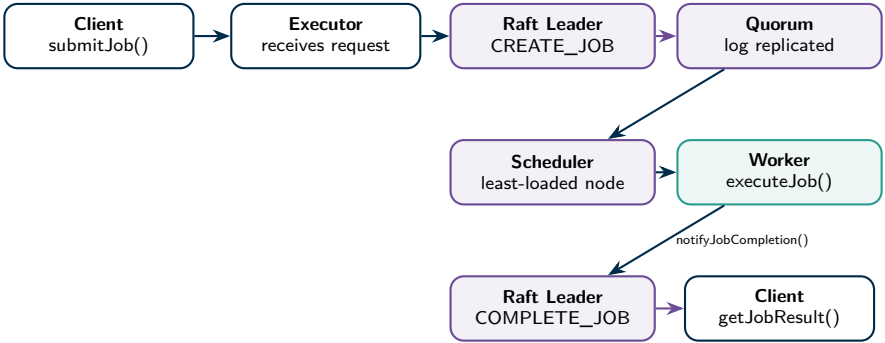
Deterministic state engine that applied committed entries sequentially.

- **Jobs Registry:** Stores active jobs in a `Map<jobId, JobMetadata>` (state, worker, attempt, result).
- **Sequential Apply:** Monotonically updates `lastAppliedIndex` up to `commitIndex`.

Job Lifecycle



End-to-End Job Submission



- The client does **not need to know the current Leader**.
- A Follower transparently forwards the request.
- The job is considered durably accepted only after the **Raft commit**.
- Execution starts only after the `ASSIGN_JOB` decision is committed.

Generic Job Execution & Code Mobility

`SerializableSupplier<T>`

A job contains executable code represented by a serializable Java functional interface. It decouple the logic of the creation, to the real retrieving of the object when needed.

The serialized `Job<T>` can be transferred through RMI to the selected executor.

Worker Execution

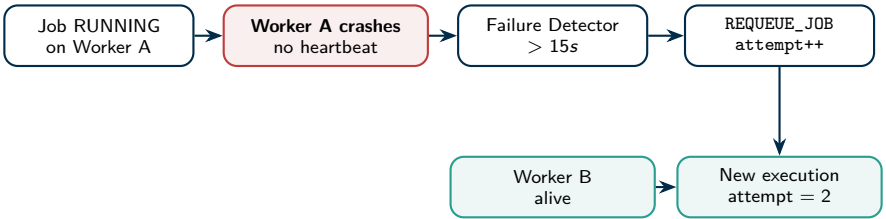
Each Executor provides:

- Local thread pool.
- Concurrent job execution.
- Active-job tracking.
- Completion notification to the Leader.

The worker does **not decide the global scheduling policy**: the Leader makes the scheduling decision and records it through Raft.

Control plane decides | **Data plane executes**

Failure Handling: Worker Crash



Stale completion protection

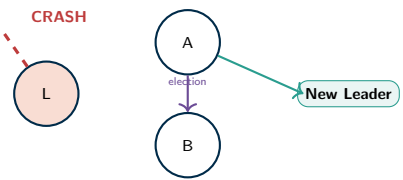
If the crashed worker later reconnects and sends the result of the old attempt, the Leader compares the reported `attempt` with the current value in the `JobStateMachine`.
An obsolete result is discarded.

Execution semantics: at-least-once with attempt-based stale-result rejection.

Failure Handling: Leader Crash

Before the Crash

- Leader has term T.
- Followers maintain compatible logs.



After the Crash

1. Followers detect the missing heartbeats from the leader.
2. Election timeout expires.
3. A candidate increments the term.
4. RequestVote is exchanged.
5. Candidate with an up-to-date log can become Leader.
6. New Leader continues from the replicated state.

The new Leader does not reconstruct the system from scratch: it continues from the replicated Raft state.

Network Partitions & Split-Brain Prevention

Majority Quorum

For a cluster of N nodes:

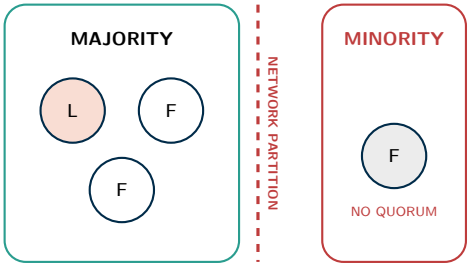
$$Q = \left\lfloor \frac{N}{2} \right\rfloor + 1$$

Only a partition containing at least Q nodes can:

- elect a Leader;
- commit new log entries;
- continue modifying replicated state.

Minority Partition

A minority partition cannot reach quorum.
Therefore it cannot safely commit conflicting decisions.



Consistency is preferred over availability when the cluster loses its majority.

Client Resilience & Request Deduplication

Transparent Failover

The client:

1. Contacts any executor.
2. Retries after a connection failure.
3. Discovers another live node.
4. Resubmits the request.

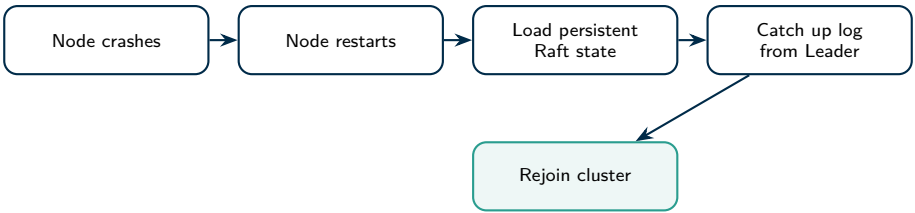
Request Identity

Every submission contains:

$$(\texttt{clientId}, \texttt{requestId})$$

The `JobStateMachine` stores previously processed client requests.
If the client retries after a timeout, the same logical request is not inserted twice into the replicated state.

Crash-Recovery & Node Rejoin



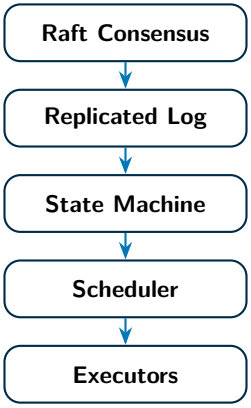
Reintegration

The restarted node can synchronize its log with the current Leader and reconstruct the application state by replaying committed entries.

Conclusions & Key Achievements

Final Architecture

- **Raft consensus** for cluster coordination.
- **Replicated persistent log** for durable decisions.
- **JobStateMachine** for deterministic application state.
- **Leader-based scheduling** with least-loaded dispatch.
- **Crash recovery** through re-election, log recovery and job requeue.
- **Code mobility** through serializable Java jobs.
- **UDP discovery** combined with reliable RMI communication.



Thank you!

Distributed Job Scheduling

